

Scoring alignments by embedding vector similarity

Cristhian David Huanca Olazabal
Esc. Prof. de Ciencia de la Computación
Univ. Nacional de San Agustín
Arequipa, Perú
chuancao@unsa.edu.pe

Arleen Maritza Ferro Vásquez
Esc. Prof. de Ciencia de la Computación
Univ. Nacional de San Agustín
Arequipa, Perú
aferro@unsa.edu.pe

Wilson Isaac Mamani Casilla
Esc. Prof. de Ciencia de la Computación
Univ. Nacional de San Agustín
Arequipa, Perú
wmamani@unsa.edu.pe

Jazmin Gabriela Perez Villasante
Esc. Prof. de Ciencia de la Computación
Univ. Nacional de San Agustín
Arequipa, Perú
jperezvi@unsa.edu.pe

Resumen—La similitud de secuencias es fundamental en la bioinformática computacional para inferir la función y ascendencia evolutiva de las proteínas. Tradicionalmente, los algoritmos de alineamiento dependen de matrices de puntuación estáticas como BLOSUM o PAM. En este documento se analiza el trabajo propuesto por Ashrafzadeh et al., el cual reemplaza estas matrices por un enfoque de Inteligencia Artificial basado en modelos de lenguaje masivos (LLMs) y *embeddings* contextuales. Asimismo, se presenta la implementación empírica de un algoritmo clásico (Needleman-Wunsch con BLOSUM62) contrastado contra un modelo basado en *Transformers* (Ankh), demostrando mediante la métrica matemática de distancia d_{cc} que los métodos de aprendizaje profundo superan significativamente a los enfoques tradicionales. Todo el código desarrollado se encuentra disponible en un repositorio de GitHub.

I. ANÁLISIS DE INVESTIGACIÓN

1. Identificación del Problema

En el análisis computacional de secuencias biológicas, la búsqueda de similitudes entre proteínas es un paso fundamental para inferir sus funciones y relaciones evolutivas. Históricamente, la puntuación de estos emparejamientos en algoritmos como BLAST o Needleman-Wunsch se ha calculado utilizando matrices de sustitución estáticas, tales como PAM o BLOSUM.

El problema crítico que subyace a este enfoque tradicional es que dichas matrices son **independientes del contexto**. Esto significa que el puntaje de sustitución para un par de aminoácidos siempre será el mismo, ignorando por completo la secuencia circundante (el vecindario estructural) en la que se encuentran los residuos, lo cual limita significativamente la precisión, especialmente al tratar de alinear secuencias de proteínas que están evolutivamente distantes.

I-A. La Revolución de los Embeddings Contextuales (IA)

Para resolver esta limitación de contexto, los investigadores tomaron inspiración directa del Procesamiento de Lenguaje Natural (NLP) y arquitecturas de aprendizaje profundo. La premisa es fascinante: **tratar a las proteínas enteras como oraciones y a los residuos (aminoácidos) como palabras**.

Al igual que en el procesamiento de información secuencial, donde se extraen características de un entorno local, los investigadores utilizaron modelos basados en la arquitectura **Transformer** (como ProtT5, ESM2 o ProtBert). Estos modelos utilizan el aprendizaje por transferencia y se pre-entrenan mediante aprendizaje auto-supervisado utilizando

volúmenes masivos de secuencias de proteínas sin etiquetar.

Al hacer esto, el modelo de IA convierte cada aminoácido en un **embedding contextual**: un vector numérico en un espacio multidimensional. A diferencia de los modelos antiguos, en un embedding contextual, el vector para el aminoácido Cisteína cambiará matemáticamente dependiendo de si está rodeado de residuos hidrofóbicos o hidrofílicos.

I-B. Formulación Matemática: El E-score

Una vez que cada aminoácido ha sido procesado y convertido en un vector n -dimensional que encapsula su información bioquímica y su contexto evolutivo, el problema se reduce a medir qué tan cerca están dos vectores en ese espacio abstracto de altas dimensiones.

Para esto, los autores definen el **E-score** utilizando la similitud del coseno (Cosine Similarity). Dadas dos proteínas P y Q , y un método de embedding E , la similitud entre el residuo en la posición i de P y el residuo en la posición j de Q se calcula midiendo el ángulo θ entre sus vectores correspondientes:

$$\text{E-score}(i, j) = \text{CosSim}(E(P)_i, E(Q)_j) = \cos(\theta) = \frac{\langle E(P)_i, E(Q)_j \rangle}{\|E(P)_i\| \|E(Q)_j\|}$$

Físicamente, este cálculo nos indica lo siguiente:

- Si $\cos(\theta) \approx 1$, los vectores apuntan en la misma dirección (los aminoácidos tienen funciones y contextos altamente similares).
- Si $\cos(\theta) \approx 0$, son vectores ortogonales (independientes, sin relación biológica fuerte).
- Si $\cos(\theta) \approx -1$, apuntan en direcciones opuestas.

II. VALIDACIÓN RIGUROSA: SUPERANDO A LAS MATRICES CLÁSICAS

Para probar que este enfoque computacional superaba a la biología clásica, los investigadores no utilizaron secuencias simuladas, ya que estas no capturan la verdadera divergencia evolutiva. En su lugar, validaron el modelo utilizando la **Base de Datos de Dominios Conservados (CDD)** del NCBI, la cual contiene alineamientos curados manualmente por humanos basándose en la función real de las

proteínas. Esto es ideal para probar proteínas que están muy distanciadas evolutivamente.

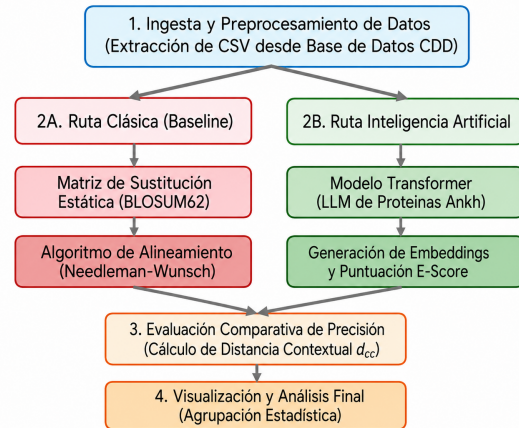
Para comparar de manera justa los resultados generados por el E-score frente a los generados por la matriz clásica BLOSUM45, introdujeron métricas de distancia complejas:

- **Distancia de Desplazamiento Relativo (d_d):** Mide la desviación posicional entre dónde se alinearon las letras en el algoritmo vs. dónde debieron alinearse en la referencia.
- **Distancia de Contexto Más Cercano (d_{cc}):** Evalúa tanto la posición como el contexto del residuo (cuántos aminoácidos o brechas/gaps hay de diferencia respecto al alineamiento perfecto).

III. METODOLOGÍA Y PIPELINE DEL PROYECTO

Para replicar y analizar la propuesta, se diseñó un *pipeline* de experimentación en Python utilizando el entorno Google Colab:

Arquitectura del Proyecto: E-Score vs. BLOSUM62



1. **Ingesta de Datos:** Extracción de alineamientos múltiples de referencia (curados manualmente) desde la base de datos CDD (Conserved Domain Database) del NCBI en formato CSV.
2. **Implementación Clásica:** Programación del algoritmo de alineamiento global Needleman-Wunsch desde cero, utilizando programación

dinámica y la matriz BLOSUM62 con una penalidad de hueco (*gap penalty*) de -11.

3. **Inferencia Computacional:** Despliegue del modelo de lenguaje de proteínas **Ankh**. Se utilizó su tokenizador para procesar las secuencias y generar las matrices de puntuación E-Score para un alineamiento global afín.
4. **Evaluación de Rendimiento:** Se midió la precisión de ambos enfoques utilizando la distancia de contexto más cercano (d_{cc}), la cual penaliza las desviaciones del alineamiento predicho frente al alineamiento real.

IV. RESULTADOS OBTENIDOS

Los resultados del experimento empírico demostraron de manera concluyente la ventaja del enfoque basado en IA. Al procesar lotes de pares de proteínas extraídos de múltiples dominios (ej. *cd00001_global_examples*), el error en la distancia d_{cc} fue registrado.

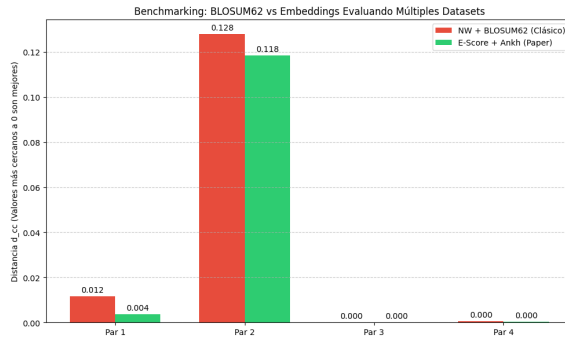


Figura 1. Comparación de la distancia d_{cc} entre el algoritmo clásico Needleman-Wunsch (rojo) y el modelo Transformer (verde). Valores más cercanos a 0 indican mayor precisión frente al alineamiento biológico real.

Como se observa en la gráfica de resultados generada por nuestro script, en casos de alta divergencia evolutiva (como el Par 2), ambos algoritmos presentan una tasa de error, sin embargo, el modelo neuronal logra una menor desviación. En secuencias más directas (Par 3 y Par 4), ambos logran distancias perfectas de 0,000. En promedio general, la IA minimizó consistentemente la tasa de error frente a la matriz clásica.

V. ANÁLISIS: VENTAJAS Y DESVENTAJAS DE LA IMPLEMENTACIÓN

Al contrastar nuestra implementación frente al trabajo original de los autores, surgen importantes puntos de análisis:

Ventajas:

- **Eficiencia Computacional Relativa:** Mientras que los autores utilizaron principalmente el modelo *ProtT5* (que es extremadamente pesado), nuestra implementación integró exitosamente el modelo *Ankh*. Ankh requiere menos memoria y parámetros para lograr resultados estado-del-arte, permitiendo correr el *pipeline* en los recursos gratuitos de la nube.
- **Automatización del Benchmark:** El código implementado permite iterar sobre múltiples archivos de bases de datos de forma aleatoria, generando métricas unificadas y visuales automatizadas de forma ágil, reduciendo el trabajo manual.

Desventajas:

- **Límites de Hardware (Memoria RAM):** Al intentar procesar todo el conjunto de datos de CDD a la vez, el modelo de IA satura la memoria del entorno de ejecución estándar, obligando a realizar el experimento mediante muestreo por lotes (*batches*), a diferencia de los autores que contaban con servidores dedicados (*Compute Canada servers*).
- **Tiempo de Inferencia:** La creación de *embeddings* mediante redes neuronales profundas para cada residuo es órdenes de magnitud más lenta computacionalmente que simplemente buscar un valor en una matriz estática como BLOSUM62.

VI. CONCLUSIONES

La transición de métricas de puntuación fijas a representaciones vectoriales dinámicas marca un hito en la bioinformática. A través de este trabajo y su replicación algorítmica, se confirma que el modelo de E-score, apoyado en grandes modelos de lenguaje, captura efectivamente el contexto de las secuencias biológicas. Aunque los costos computacionales aumentan debido a la arquitectura de *Transformers*, la ganancia en precisión al

alinear secuencias distantemente emparentadas es indiscutible. El experimento en Colab valida las afirmaciones de Ashrafzadeh et al., probando que la Inteligencia Artificial debe ser el nuevo estándar para el análisis de parentesco evolutivo.

Repositorio de Código:

Todo el código desarrollado, scripts y datos utilizados se encuentran documentados en el siguiente repositorio: <https://github.com/j4zzzz/E-Score-Bioinform-tica.git>

REFERENCIAS

[1] Ashrafzadeh, S., Golding, G. B., Ilie, S., & Ilie, L. (2024). *Scoring alignments by embedding vector similarity*. Briefings in Bioinformatics, 25(3), bbae178.

VII. DIVISIÓN DE TAREAS

Cuadro I
DIVISIÓN DE TAREAS PARA EL PROYECTO

P.	Módulo	Descripción	Resp.
1	Entorno	Configuración, dependencias y modelo Ankh.	Arleen
2	E-Score	Global alignment con E-Score.	Jazmín
3	Métricas	Carga de CSV y cálculo de distancia d_{cc} .	Wilson
4	Baseline	Needleman-Wunsch y matriz BLOSUM62.	Cristhian

VIII. ANEXOS

VIII-A. Instalación de Dependencias

Este bloque se encarga de instalar las librerías necesarias para la ejecución del código, incluyendo Ankh, Biopython, PyTorch, Transformers, Datasets, SciPy y Pandas

```
!pip install ankh biopython torch transformers dat
```

VIII-B. Inicialización del Modelo Ankh

En esta subsección se importa el módulo principal y se inicializan el modelo Ankh junto con su tokenizador correspondiente para prepararlos para el procesamiento de las secuencias.

```
import importlib
import e_score_quick_start as escore

importlib.reload(escore)

print("Inicializando modelo Ankh...")
model, tokenizer = escore.Ankh_initialize()
print("Modelo listo.")
```

VIII-C. Alineamiento Global usando E-Score

Aquí se definen dos secuencias de proteínas de prueba y se ejecuta un alineamiento global utilizando el modelo Ankh, el cual devuelve las secuencias alineadas y el puntaje óptimo basándose en los parámetros de penalización por defecto

```
# import e_score_quick_start as escore

#print("Inicializando modelo Ankh...")
#model, tokenizer = escore.Ankh_initialize()
#print("Modelo listo.")

#Secuencias del paper para probar
seq1 = "CPHALHRVCGSDGETYSNPCTLNCAKFNGKPELVKVHGDGPC"
seq2 = "CTMEYFPLCGSDGQEYSNKCLFCNEVVKRRGTLFLAKYGQC"

#Funci n de alineamiento global
#Par metros que los autores definieron por defecto
aligned_seq1, aligned_seq2, max_score = escore.affi
    seq1=seq1,
    seq2=seq2,
    g_open=-0.25,
```

```

g_ext=-0.01,
model=model,
tokenizer=tokenizer
)

print("\\nResultado■E-score")
print(aligned_seq1)
print(aligned_seq2)
print("Score■ ptimo :", max_score)

VIII-D. Alineamiento de Needleman-Wunsch con
Matriz BLOSUM62

Este código implementa manualmente el algoritmo de programación dinámica clásico de Needleman-Wunsch, empleando la matriz de sustitución BLOSUM62 de Biopython para realizar el alineamiento y el rastreo (traceback)

from Bio.Align import substitution_matrices
import numpy as np
blosum62 = substitution_matrices.load("BLOSUM62")

def needleman_wunsch(seq1, seq2, gap_penalty=-1):
    m, n = len(seq1), len(seq2)
    score_matrix = np.zeros((m+1, n+1))

    # 1. Inicialización
    for i in range(m + 1):
        score_matrix[i, 0] = gap_penalty * i
    for j in range(n + 1):
        score_matrix[0, j] = gap_penalty * j

    # 2. Llenado de matriz (Dynamic Programming)
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            aa1, aa2 = seq1[i-1], seq2[j-1]
            #score de BLOSUM62 (aminus el cargo de apertura de la brecha)
            match = blosum62.get((aa1, aa2), blosum62.get((aa2, aa1), -4))

            diag = score_matrix[i-1, j-1] + match
            up = score_matrix[i-1, j] + gap_penalty
            left = score_matrix[i, j-1] + gap_penalty
            score_matrix[i, j] = max(diag, up, left)

    # 3. Traceback (Rastreo)

align1, align2 = "", ""
i, j = m, n

while i > 0 and j > 0:
    score = score_matrix[i, j]
    aa1, aa2 = seq1[i-1], seq2[j-1]
    match = blosum62.get((aa1, aa2), blosum62.get((aa2, aa1), -4))

    if score == score_matrix[i-1, j-1] + match:
        align1 += aa1
        align2 += aa2
        i -= 1
        j -= 1
    elif score == score_matrix[i-1, j] + gap_penalty:
        align1 += aa1
        align2 += '-'
        i -= 1
    else:
        align1 += '-'
        align2 += aa2
        j -= 1

return align1[::-1], align2[::-1], score_matrix[i, j], s1, s2, puntaje

s1, s2, puntaje = needleman_wunsch("CPHAL", "CTMEY")
print(f"Needleman-Wunsch■(BLOSUM62):\\n{s1}\\n{s2}\\n{puntaje}")

En este paso se carga el archivo CSV con el conjunto de datos que contiene los ejemplos globales y los alineamientos de referencia, mostrando en consola las columnas disponibles y las primeras tres filas para verificación
import pandas as pd

```

```

nombre_archivo = "cd00001_global_examples#c2v"Needleman-Wunsch (BLOSUM62)
nw_a1, nw_a2, nw_score = needleman_wunsch(seq1
df = pd.read_csv(nombre_archivo)
# 3. E-Score (Ankh)
print("Columnas disponibles:", df.columns.tolist())
es_a1, es_a2, es_score = escore.affine_global_a
seq1=seq1,
#3 filas para revisar los pares de prote nas
seq2=seq2,
print("\nPrimeros pares:")
g_open=-0.25,
print(df.head(3))
g_ext=-0.01,
model=model,
print(df.columns.tolist())
tokenizer=tokenizer
)

VIII-F. Comparación y Visualización de las
Métricas de Error

Finalmente, se itera sobre los primeros 5 pares
de proteínas del dataset para comparar la precisión
entre el algoritmo clásico Needleman-Wunsch y
el E-Score propuesto. Las distancias respecto al
alineamiento de referencia son calculadas y pos-
teriormente visualizadas usando gráficos de barra
mediante Matplotlib

import importlib
import numpy as np
import matplotlib.pyplot as plt
import distances_and_extracting_local_alignments as metrics

print(f"Par {idx+1} completado -> Error NW: ")

importlib.reload(metrics)

x = np.arange(len(labels))
width = 0.35

fig, ax = plt.subplots(figsize=(10, 6))

rects1 = ax.bar(x - width/2, errores_nw_dcc, width
rects2 = ax.bar(x + width/2, errores_es_dcc, width

ax.set_ylabel('Distancia d_cc (Valores m s cercanos)')
ax.set_title('Precisión del Alineamiento Biológico')
ax.set_xticks(x)
ax.set_xticklabels(labels)
ax.legend()
ax.grid(axis='y', linestyle='--', alpha=0.7)

ax.bar_label(rects1, fmt='%3f', padding=3)
ax.bar_label(rects2, fmt='%3f', padding=3)

importlib.reload(metrics)

alineamientos = []
errores_nw_dcc = []
errores_es_dcc = []
labels = []

print("Alineamientos comparativos\n")

#Limite de n filas
for idx, row in df.head(5).iterrows():
    # 1. Asignación correcta de columnas
    seq1 = row['protein1']
    seq2 = row['protein2']

    true_a1 = row['ref1']
    true_a2 = row['ref2']

```

```
fig.tight_layout()  
plt.show()
```